

Связи между таблицами: практика

8 заданий: от одной связи «один ко многим» до подсчётов по таблице-связке и схемы базы в draw.io

Задания 1–7 — SQL на учебной базе колледжа: группы (`groups`), студенты (`students`), курсы (`courses`) и записи на курсы (`enrollments`). **Задание 8** — проектирование: схема связей таблиц интернет-магазина в draw.io.

- В каждом SQL-задании нужно написать **один** запрос указанного типа. Перед заданием показано, какие таблицы и данные уже есть в базе.
- Проверка внешних ключей включена (`PRAGMA foreign_keys = ON`).
- SQL-задания можно решить и проверить в SQL-тренажёре: тема «Связи между таблицами».

1 Один ко многим: студенты и группы

[CREATE TABLE](#)

В базе уже есть таблица `groups` (`id`, `name`). В одной группе учится много студентов, а каждый студент — ровно в одной группе: это связь **«один ко многим»**.

Создайте таблицу `students` со столбцами:

- `id` `INTEGER PRIMARY KEY`
- `name` `TEXT NOT NULL`
- `group_id` `INTEGER NOT NULL` — внешний ключ на `groups(id)`

Подумайте, в какой таблице должен жить внешний ключ: у «многих» (студентов) или у «одного» (группы)?

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

`groups`

`id` `INTEGER PK`, `name` `TEXT`

id	name
1	WEB-21
2	IB-22

Ваш SQL-запрос

Есть таблицы `students` и `courses`. Один студент может записаться на несколько курсов, а на одном курсе учиться много студентов — это связь **«многие ко многим»**. Одним внешним ключом её не выразить, нужна промежуточная таблица.

Создайте таблицу `enrollments` (записи на курсы):

- `student_id` `INTEGER NOT NULL` — внешний ключ на `students(id)`
- `course_id` `INTEGER NOT NULL` — внешний ключ на `courses(id)`
- составной первичный ключ `PRIMARY KEY (student_id, course_id)` — чтобы одного студента нельзя было записать на один курс дважды.

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

groups

`id` `INTEGER` PK, `name` `TEXT`

id	name
1	WEB-21
2	IB-22

students

`id` `INTEGER` PK, `name` `TEXT`, `group_id` `INTEGER` → `groups.id`

id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2
4	Севара	2
5	Даниил	1

courses

`id` `INTEGER` PK, `title` `TEXT`

id	title
1	Python
2	Базы данных
3	JavaScript

Ваш SQL-запрос

3 Запись на курс по идентификаторам

INSERT

Севара хочет записаться на курс «Базы данных». В таблицу-связку пишутся не имена и названия, а **идентификаторы**.

Посмотрите в таблицах `students` и `courses`, какие `id` у Севары и у курса «Базы данных», и добавьте одну строку в `enrollments` (`student_id`, `course_id`).

Проверка внешних ключей включена: если указать несуществующий `id`, база выдаст ошибку `FOREIGN KEY constraint failed`.

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

groups

`id` INTEGER PK, `name` TEXT

id	name
1	WEB-21
2	IB-22

students

`id` INTEGER PK, `name` TEXT, `group_id` INTEGER → `groups.id`

id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2
4	Севара	2
5	Даниил	1

courses

`id` INTEGER PK, `title` TEXT

id	title
1	Python
2	Базы данных
3	JavaScript

enrollments

`student_id` INTEGER PK → `students.id`, `course_id` INTEGER PK → `courses.id`

student_id	course_id
1	1
1	2
2	3
3	1
3	2
5	1

Ваш SQL-запрос

4 Кто учится на курсе Python

SELECT

Выведите столбец `name` всех студентов, записанных на курс **Python**.

Цепочка такая: название курса → его `id` в `courses` → `student_id` в `enrollments` → имена в `students`. Используйте вложенные запросы и оператор `IN`, потому что студентов несколько.

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

groups

id INTEGER PK, name TEXT

id	name
1	WEB-21
2	IB-22

students

id INTEGER PK, name TEXT, group_id INTEGER → groups.id

id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2
4	Севара	2
5	Даниил	1

courses

id INTEGER PK, title TEXT

id	title
1	Python
2	Базы данных
3	JavaScript

enrollments

student_id INTEGER PK → students.id, course_id INTEGER PK → courses.id

student_id	course_id
1	1
1	2
2	3
3	1
3	2
5	1

Ваш SQL-запрос

5 Один к одному: студенческий билет

[CREATE TABLE](#)

У каждого студента ровно один студенческий билет, и каждый билет принадлежит ровно одному студенту — это связь **«один к одному»**.

Создайте таблицу `student_cards`:

- `id` `INTEGER PRIMARY KEY`
- `student_id` `INTEGER NOT NULL UNIQUE` — внешний ключ на `students(id)`
- `card_number` `TEXT NOT NULL`

Именно `UNIQUE` на внешнем ключе превращает «один ко многим» в «один к одному»: второй билет для того же студента база не примет.

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

groups

`id` `INTEGER PK`, `name` `TEXT`

id	name
1	WEB-21
2	IB-22

students

`id` `INTEGER PK`, `name` `TEXT`, `group_id` `INTEGER` → `groups.id`

id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2
4	Севара	2
5	Даниил	1

courses

`id` `INTEGER PK`, `title` `TEXT`

id	title
1	Python
2	Базы данных
3	JavaScript
4	Алгоритмы

enrollments

`student_id` `INTEGER PK` → `students.id`, `course_id` `INTEGER PK` → `courses.id`

student_id	course_id
1	1
1	2
2	3
3	1
3	2
5	1

Ваш SQL-запрос

6 Курсы, на которые никто не записан

SELECT

Учебная часть хочет закрыть пустые курсы. Выведите столбец `title` курсов, на которые **не записан ни один** студент.

Подсказка: курс пустой, если его `id` **нет** среди `course_id` в `enrollments` — используйте `NOT IN` с вложенным запросом.

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

groups

id INTEGER PK, name TEXT

id	name
1	WEB-21
2	IB-22

students

id INTEGER PK, name TEXT, group_id INTEGER → groups.id

id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2
4	Севара	2
5	Даниил	1

courses

id INTEGER PK, title TEXT

id	title
1	Python
2	Базы данных
3	JavaScript
4	Алгоритмы

enrollments

student_id INTEGER PK → students.id, course_id INTEGER PK → courses.id

student_id	course_id
1	1
1	2
2	3
3	1
3	2
5	1

Ваш SQL-запрос

Выведите столбец `name` студентов, которые записаны **на два курса или больше**.

Сначала во вложенном запросе сгруппируйте `enrollments` по `student_id` и оставьте группы с `HAVING COUNT(*) >= 2`, затем найдите имена этих студентов через `IN`.

ЧТО УЖЕ ЕСТЬ В БАЗЕ ДАННЫХ

groups

id INTEGER PK, name TEXT

id	name
1	WEB-21
2	IB-22

students

id INTEGER PK, name TEXT, group_id INTEGER → groups.id

id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2
4	Севара	2
5	Даниил	1

courses

id INTEGER PK, title TEXT

id	title
1	Python
2	Базы данных
3	JavaScript
4	Алгоритмы

enrollments

student_id INTEGER PK → students.id, course_id INTEGER PK → courses.id

student_id	course_id
1	1
1	2
2	3
3	1
3	2
5	1

Ваш SQL-запрос

Интернет-магазину нужна база данных для каталога товаров. В ней три таблицы: **категории**, **поставщики** и **товары**. Ниже описаны их столбцы и связи.

Задача: нарисуйте ER-диаграмму (схему связей) этих таблиц в **draw.io** (сайт app.diagrams.net) или в похожей программе для схем.

ХАРАКТЕРИСТИКИ ТАБЛИЦ

categories – Категории товаров

Справочник категорий каталога: «Электроника», «Мебель», «Книги»... Каждая категория записана один раз; товары ссылаются на неё по id.

Столбец	Тип	Ограничения	Описание
id	INTEGER	PRIMARY KEY	Уникальный номер категории
name	TEXT	NOT NULL, UNIQUE	Название категории — не может повторяться
description	TEXT	—	Краткое описание категории (может быть пустым)

Пример строки: (1, 'Электроника', 'Компьютеры, телефоны и аксессуары')

suppliers – Поставщики

Компании, которые привозят товары в магазин. Один поставщик может поставлять много разных товаров.

Столбец	Тип	Ограничения	Описание
id	INTEGER	PRIMARY KEY	Уникальный номер поставщика
name	TEXT	NOT NULL	Название компании
phone	TEXT	—	Контактный телефон
city	TEXT	NOT NULL	Город, из которого идут поставки

Пример строки: (1, 'ООО «ТехноОпт»', '+998 71 123-45-67', 'Ташкент')

products – Товары

Каталог товаров. У каждого товара ровно одна категория и ровно один поставщик — они хранятся не названиями, а ссылками (внешними ключами).

Столбец	Тип	Ограничения	Описание
id	INTEGER	PRIMARY KEY	Уникальный номер товара
name	TEXT	NOT NULL	Название товара
price	REAL	NOT NULL, CHECK (price > 0)	Цена за единицу
quantity	INTEGER	NOT NULL, DEFAULT 0	Остаток на складе
category_id	INTEGER	NOT NULL, FK → categories.id	Категория товара
supplier_id	INTEGER	NOT NULL, FK → suppliers.id	Поставщик товара

Пример строки: (1, 'Ноутбук', 55000, 10, 1, 1)

ОПИСАНИЕ СВЯЗЕЙ

Связь	Тип	Где внешний ключ	Смысл
categories → products	один ко многим (1 : N)	products.category_id	В одной категории много товаров, у товара одна категория
suppliers → products	один ко многим (1 : N)	products.supplier_id	Поставщик привозит много товаров, у товара один поставщик

КАК НАРИСОВАТЬ В DRAW.IO

1. Откройте `app.diagrams.net` → «Create New Diagram» → «Blank Diagram».
2. Слева в библиотеке фигур включите раздел **Entity Relation** («Ещё фигуры» → Entity Relation). Перетащите фигуру **Table** для каждой из трёх таблиц.
3. В каждой таблице: заголовок — имя таблицы, строки — столбцы с типами. Отметьте первичный ключ меткой **PK**, внешние ключи — меткой **FK**.
4. Соедините таблицы линиями **от внешнего ключа к первичному ключу**:
`products.category_id` → `categories.id` и `products.supplier_id` → `suppliers.id`.
5. Для линий выберите стрелки нотации «гусиная лапка» (crow's foot): со стороны `categories` и `suppliers` — «ровно один», со стороны `products` — «много». Подпишите на линиях **1** и **N**.
6. Под схемой добавьте текстовый блок: 2–3 предложения о том, зачем категории и поставщики вынесены в отдельные таблицы.
7. Сохраните файл как `.drawio` и экспортируйте в `.png` (File → Export as → PNG).

Подсказка: в draw.io есть Arrange → Insert → Advanced → SQL. Если вставить туда CREATE TABLE этих таблиц, программа сама нарисует заготовки таблиц. Связи и подписи всё равно нужно добавить вручную.

ЧТО СДАТЬ И КАК БУДЕТ ПРОВЕРЯТЬСЯ

- ☐ Есть все три таблицы со всеми столбцами и типами из характеристик выше.
- ☐ Первичные ключи отмечены PK, внешние ключи — FK.
- ☐ Две связи нарисованы от внешнего ключа к первичному ключу, с обозначением «1» и «N» (или «гусиными лапками»).
- ☐ Нет лишних связей: categories и suppliers между собой не связаны.
- ☐ Есть короткое описание: зачем данные разделены на три таблицы.
- ☐ Сданы два файла: `.drawio` и `.png`.

ДОПОЛНИТЕЛЬНО (ПО ЖЕЛАНИЮ)

Добавьте на схему четвёртую таблицу `orders` (заказы) и таблицу-связку `order_items` (товары в заказе: `order_id`, `product_id`, `quantity`). Покажите связь «многие ко многим» между заказами и товарами.